

Software for Mobile Devices (Assignment 1)

Open-Ended, AI-Assisted Application Development Assignment

React Native • JavaScript • React Concepts • 100 Marks

1. Assignment Overview

AI tools and coding agents are permitted. You may use AI for ideation, code generation, debugging, learning, refactoring, documentation, and other development activities. However, the submitted application must be understood by you and defended during the viva.

2. Reimagine the University Student Portal (FLEX)

Develop a mobile-first application that addresses a real problem, inconvenience, or opportunity experienced by university students. The application may improve an existing student portal experience or solve a different student life/academic problem.

There is no single correct application. You are encouraged to interpret the problem creatively and propose their own solution.

3. Possible Problem Areas

- Student academic dashboard or personalized portal
- Attendance monitoring and academic alerts
- Course Registration
- View Course Wise Marks
- Transcript
- Fee Challan / Details
- Student/Course feedback
- Peer study/resource sharing

These are examples, not mandatory modules. You may replace, combine, extend, or redesign them.

4. Core Requirements

Every submission should demonstrate the following, regardless of the chosen application idea:

Requirement	Description
A. Meaningful Problem	Clearly identify a real user problem or opportunity and explain how the application addresses it.
B. Mobile Application	The solution must be implemented as a React Native mobile application. (Do not use any side or bottom bars as we have not discussed anything in the class)
C. React Concepts	Demonstrate meaningful use of React concepts such as components, props, state, events, conditional rendering, and data-driven UI.
D. JavaScript	Use JavaScript concepts appropriately, including

	arrays/objects, functions, array methods, conditions, and data manipulation where relevant.
E. User Interaction	Users must be able to interact with the application; it should not be only a collection of static screens. (Focus on functional, interactive UI improvements over static screen replication, we do not need a replica of the Flex rather a well-designed and interactive solution)
F. Data-Driven UI	At least part of the interface should be generated from application data rather than manually duplicated UI elements (create Objects and arrays for static data)
G. Form/Input	Include at least one meaningful user input or form with appropriate validation/feedback where applicable. (look for the built in React native component properties)
H. Application States	Handle relevant states such as empty data, invalid input, different data values, or changing application state.
I. Reusable Components	Use reusable components where repetition or shared UI behavior makes componentization appropriate.
J. Usability	The application should be understandable, consistent, and reasonably easy to use on a mobile device. (important)

5. Advanced Features

You should choose features that genuinely improve their solution rather than adding features only to increase the number of screens.

- Search and filtering
- Sorting and prioritization
- Personalized dashboard
- Dynamic warnings or recommendations
- Multiple user/application states
- Attendance and grade calculations
- Empty states and error states
- Data validation and helpful feedback
- Interactive cards, lists, or summaries
- Meaningful animations or interaction feedback (you can use any third-party package or library)
- Any original feature that strengthens the proposed solution

6. Data-Driven and Dynamic Behavior

The application should respond appropriately when its underlying data changes. For example, changing attendance, adding a course, changing an assignment deadline, or having no announcements should result in appropriate UI behavior.

The purpose is to demonstrate understanding of state, data manipulation, conditional rendering, and reusable UI (not to require a specific data source or library).

7. Dashboard Requirement

You must create a **dashboard** in their application using [react-native-chart-kit](#).

The dashboard should present meaningful information through **at least two different types of charts**, such as Line Chart, Bar Chart, Pie Chart, or Progress Chart.

You are free to decide:

- What data to display
- Which chart types to use
- How to arrange the dashboard
- The visual design and theme

The purpose is to evaluate whether you can **use data effectively and design a useful dashboard**, rather than simply displaying charts.

8. AI-Assisted Development Policy

AI tools and AI coding agents are allowed and encouraged. You are responsible for reviewing, testing, understanding, and adapting AI-generated output.

- You may use ChatGPT, Gemini, Claude, Copilot, Codex, or other AI development tools.
- AI may be used for brainstorming, UI ideas, code generation, debugging, explanations, refactoring, and documentation.
- You must not assume that AI-generated code is correct; they are responsible for testing and validating it.
- You should be able to explain important parts of their submitted application during the viva.
- The amount of AI-generated code is not a grading criterion. Quality, understanding, decisions, and adaptability are.

9. Viva and Live Adaptation

Each student will have at least 5-minute viva. The purpose is to assess understanding, engineering decisions, and adaptability (not memorization).

Viva Component	What is assessed	Approx. Time
Application structure	Explain the main components and how the application is organized.	1 min
React understanding	Explain where and why state, props, events, or conditional rendering are used.	1 min
JavaScript/data handling	Explain a relevant section involving arrays, objects, filtering, mapping, conditions, or calculations.	1 min
Live modification	Make a small change requested by the instructor, such as changing a threshold, adding data, changing a filter, or modifying behavior.	1 min
Design/AI decision	Defend one design or implementation decision and explain how AI contributed to development.	1 min

Examples of live changes: change an attendance threshold; add a new course; display only low-attendance courses, change sorting behavior, handle an empty list, modify a condition, or change a displayed value.

10. Open-Ended Assessment Rubric — 100 Marks

Evaluation Area	Marks	What is Evaluated
Solution Quality	5	How well the student identifies and creates a meaningful solution to the intended problem.
Creativity & Innovation	15	Originality of the idea, features, interactions, presentation, and overall approach. Creativity should contribute to the solution rather than merely add decoration.
UI/UX & Usability	10	Ease of use, clarity, consistency, visual hierarchy, mobile suitability, feedback, and overall user experience.
Application Functionality	10	Correctness, completeness, reliability, and coherence of the implemented experience.
Use of JavaScript & React Concepts	10	Appropriate and meaningful use of state, props, components, events, conditional rendering, lists, array/object manipulation, forms, and related concepts covered in class.
Component Design & Code Quality	15	Reusability, organization, maintainability, sensible separation of concerns, meaningful naming, and avoidance of unnecessary duplication.
Data & Application States	10	Handling of changing values, empty data, invalid input, alternative conditions, and other relevant states.
AI-Assisted Development & Critical Thinking	5	Effective use of AI as a development aid, with evidence of review, adaptation, testing, and critical decision-making.
Viva & Adaptability	20	Ability to explain the submitted work and perform or reason through a small change requested during the viva.
TOTAL	100	Overall quality of the solution.

11. Important Grading Principles

- There is no single correct application design or feature set.
- Adding the navigation code or unnecessary code will lead to negative marking. **(you should switch views as discussed in the class)**
- Adding unnecessary features does not automatically increase marks.
- A small, well-designed application may score higher than a large but poorly designed application.
- You are evaluated on how effectively their chosen features solve the identified problem.
- AI-generated code does not receive marks by itself.
- You are responsible for the correctness and understanding of their submitted work.
- Creativity is encouraged, but creativity should support usability and problem-solving.
- The instructor/TA may ask a student to make a small change to the submitted application during the viva.

12. Submission Deliverables

- Expo code/Snack project source code.
- README/documentation containing the proposed solution, major features, and setup/run instructions.

- Screenshots or a short demonstration video.
- AI Usage Report. (template is attached) – In case of no AI use, you will be given extra credit for it.
- Push the project source code and relevant documentation to the GitHub and share the link to the GCR.

Note: If you are submitting your assignment early, please keep your repository **private**. Once the submission deadline has passed, you may make the repository public.

13. Deadline

20-Sept-2026 (Sunday) - 11:50pm

Late submission will not be entertained. Do not ask for deadline extension.

Plagiarized assignment will be marked as zero for both the students.